

TMS — Enterprise Ticket Management System

SaaS Product & Module-Wise Feature Specification

Prepared for WMX Solutions

July 2026

Contents

1. Purpose of This Document	2
2. Target Architecture	2
2.1 Why shared-database, TenantId-column multi-tenancy	2
2.2 Scaling considerations so 100+ clients don't break the system	3
3. Multi-Tenant Data Model (High Level)	4
4. Product Modules (Tenant-Facing)	5
Module 1 — Authentication & Identity	5
Module 2 — Tenant Onboarding & Workspace Setup	5
Module 3 — Ticketing Core	5
Module 4 — SLA Management	5
Module 5 — Workflow Automation & Business Rules	5
Module 6 — Knowledge Base / Self-Service Portal	6
Module 7 — Customer/End-User Portal	6
Module 8 — Notifications & Communication	6
Module 9 — Reporting & Analytics	6
Module 10 — Asset Management / CMDB (Enterprise add-on)	6
Module 11 — Integrations & Public API	6
Module 12 — Roles & Permissions (Tenant-Level RBAC)	6
5. Super Admin Module (Platform Owner Console)	7
5.1 Tenant (Client) Lifecycle Management	7
5.2 Plans & Billing Administration	7
5.3 Platform Operations & Health	7
5.4 Security & Compliance	7
5.5 Support & Success Tooling	7
5.6 Platform Admin Roles	8
6. How a Client Gets Onboarded and Managed (End-to-End Flow)	9
7. Non-Functional Requirements	10
8. Suggested Implementation Roadmap	10

1. Purpose of This Document

This document is the implementation spec for **TMS**, a multi-tenant, enterprise-grade Ticket Management System sold as SaaS. It is written to be handed module-by-module to an AI coding assistant (Claude) or a dev team, so each module can be built, reviewed, and shipped independently while staying consistent with the overall architecture.

It covers: target architecture, the multi-tenancy model, every product module with its features, the Super Admin console in detail, how clients (tenants) are onboarded and managed, the data model at a high level, non-functional requirements for scaling to 100+ clients without breaking, and a phased implementation roadmap.

2. Target Architecture

Layer	Choice	Notes
Frontend	Node.js (Next.js, React)	Deployed on Vercel — edge caching, preview deployments per PR, ISR for public status/KB pages
Backend API	.NET 8 (ASP.NET Core Web API)	Deployed on Azure App Service (or AKS at higher scale)
Database	Neon Postgres — single database TMS, shared schema, TenantId on every tenant-scoped table	Neon's serverless branching is used for per-PR / staging DB branches
Cache / Queue	Redis (Azure Cache for Redis) + background jobs via Hangfire or Azure Functions	Rate limiting, session cache, SLA timers, async email/notification dispatch
Auth	OAuth2/OIDC via ASP.NET Identity + JWT, optional SSO (SAML/Entra ID/Okta) per tenant	Platform-level auth for super admin is fully separate from tenant auth
File storage	Azure Blob Storage	Ticket attachments, KB media, tenant logos
CI/CD	GitHub Actions → Vercel (frontend) + Azure DevOps/GitHub Actions → Azure (backend)	Separate pipelines, gated by environment (dev/staging/prod)
Observability	Azure Application Insights + Sentry (frontend)	Per-tenant usage metrics feed the Super Admin analytics module

2.1 Why shared-database, TenantId-column multi-tenancy

With Neon, all tenant data lives in **one Postgres database named TMS**, and every tenant-owned table (tickets, users, categories, SLAs, etc.) carries a `TenantId` column plus a composite index (`TenantId, Id`). Row-level isolation is enforced in two layers so a bug in one layer can't leak data:

1. **Application layer** — a `TenantContext` (resolved from the authenticated user's JWT claim or subdomain) is injected into every EF Core query via a global query filter (`HasQueryFilter(t => t.TenantId == _tenantContext.TenantId)`).

2. **Database layer** — Postgres **Row-Level Security (RLS)** policies on every tenant table as a second line of defense, keyed off a session variable (`SET app.tenant_id = '...'`) set per request/connection.

This model is the cheapest to run and easiest to operate at 100+ clients (one DB to patch, back up, and monitor), while still giving hard isolation guarantees. If a specific enterprise client later demands full physical isolation for compliance, that tenant can be “promoted” to its own Neon project/database without changing the application code — only the connection resolution layer changes.

2.2 Scaling considerations so 100+ clients don’t break the system

- **Connection pooling:** Neon’s built-in pooler (PgBouncer, transaction mode) is used since .NET + EF Core opens many short-lived connections; without it, 100+ tenants each spinning up worker instances will exhaust Postgres connection limits.
- **Indexing discipline:** every query must be filtered by `TenantId` first; all indexes are composite starting with `TenantId`.
- **Noisy-neighbor protection:** per-tenant API rate limiting (ASP.NET rate limiting middleware) and per-tenant background job quotas so one client’s automation storm can’t degrade others.
- **Horizontal scaling:** the .NET API is stateless (sessions/JWT only) so Azure App Service can scale out via autoscale rules (CPU/queue-length triggers) without sticky sessions.
- **Read/write separation (future):** Neon read replicas can offload reporting/dashboard queries once tenant count grows past the point a single primary handles comfortably.
- **Background work isolation:** SLA timers, email dispatch, and report generation run in a separate worker process/queue, not inline in the request path, so a spike in one tenant’s ticket volume doesn’t slow down API latency for others.
- **Tenant-aware caching:** Redis keys are always prefixed by `TenantId` to avoid cross-tenant cache collisions.

3. Multi-Tenant Data Model (High Level)

Table	Key columns	Notes
Tenants	Id, Name, Subdomain, PlanId, Status, CreatedAt, TrialEndsAt	Root entity; everything else references it
Plans	Id, Name, MaxAgents, MaxTicketsPerMonth, PriceMonthly, Features(json)	Drives billing + feature flags
Users	Id, TenantId, Email, PasswordHash/SSOID, RoleId, Status	Tenant-scoped; a super-admin user table is separate (PlatformUsers)
Roles / Permissions	Id, TenantId (nullable for system roles), Name, PermissionSet	Supports custom roles per tenant
Tickets	Id, TenantId, Subject, Description, StatusId, PriorityId, CategoryId, RequesterId, AssigneeId, SlaPolicyId, CreatedAt, DueAt	Core entity
TicketComments / TicketAttachments	Id, TenantId, TicketId, ...	
SlaPolicies	Id, TenantId, Name, ResponseTargetMins, ResolutionTargetMins, BusinessHoursId	
Categories, Tags	Id, TenantId, Name	
Workflows / AutomationRules	Id, TenantId, TriggerType, Conditions(json), Actions(json)	No-code rule engine
KnowledgeBaseArticles	Id, TenantId, Title, Body, Visibility	Public/internal
Notifications	Id, TenantId, UserId, Channel, TemplateId, Status	
AuditLogs	Id, TenantId (nullable for platform events), ActorId, Action, EntityType, EntityId, Timestamp	Immutable, append-only
Subscriptions / Invoices	Id, TenantId, PlanId, Status, StripeCustomerId, PeriodStart/End	Billing
FeatureFlags	Id, TenantId (nullable = global default), FlagKey, Enabled	Controlled from Super Admin

4. Product Modules (Tenant-Facing)

Each module below is scoped so it can be handed to Claude as an independent implementation ticket, with a note on what “done” looks like.

Module 1 — Authentication & Identity

- Email/password signup + login, email verification, password reset.
- MFA (TOTP) optional per tenant, enforceable by tenant admin policy.
- SSO: SAML 2.0 and OIDC (Entra ID, Okta, Google Workspace) configurable per tenant.
- Session management: JWT access token + refresh token rotation, device/session list with remote revoke.
- **Done when:** a tenant admin can enable SSO for their org, users can log in via SSO or local credentials, and sessions are revocable.

Module 2 — Tenant Onboarding & Workspace Setup

- Self-serve signup (trial) and admin-provisioned signup (sales-assisted).
- Setup wizard: company name, subdomain (`acme.tms.app`), timezone, branding (logo, color), initial admin user.
- Plan selection at signup, trial countdown, upgrade/downgrade flow.
- **Done when:** a new company can go from signup to a working workspace with an admin user in under 5 minutes, unassisted.

Module 3 — Ticketing Core

- Ticket CRUD with statuses (New, Open, Pending, Resolved, Closed — customizable per tenant), priorities, categories/subcategories, custom fields (text, dropdown, date, number).
- Multi-channel intake: web portal form, email-to-ticket, API, and optionally chat widget.
- Attachments (Azure Blob), internal notes vs. public replies, merge/split tickets, linked tickets (parent/child, related).
- Full-text search and saved filters/views per agent.
- **Done when:** an agent can receive a ticket from 3 channels (portal, email, API), work it with custom fields, and close it with full history retained.

Module 4 — SLA Management

- SLA policies (response/resolution targets) by priority, category, or customer tier; business-hours calendars per tenant/region.
- Automatic SLA breach warnings and escalation triggers.
- SLA compliance reporting per tenant, per agent, per client.
- **Done when:** a breaching ticket auto-escalates and shows on the SLA dashboard before and after breach.

Module 5 — Workflow Automation & Business Rules

- No-code rule builder: “when X happens, do Y” (trigger → conditions → actions).
- Triggers: ticket created/updated, SLA about to breach, keyword in description, customer reply received.
- Actions: auto-assign (round robin / load-based), change status/priority, notify, run webhook.
- Approval workflows for tickets requiring sign-off (e.g., change requests).
- **Done when:** a tenant admin can build a rule with no code and see it fire correctly in the audit log.

Module 6 — Knowledge Base / Self-Service Portal

- Public and internal-only articles, categories, versioning, article-level analytics (views, “was this helpful”).
- Suggested articles shown to end users before they submit a ticket (deflection).
- **Done when:** an end user typing a subject line sees relevant KB suggestions before submitting.

Module 7 — Customer/End-User Portal

- Branded per-tenant portal (custom domain optional) where end users submit and track tickets.
- Ticket status tracking, comment thread, satisfaction survey (CSAT) on close.
- **Done when:** an end user can log in to a tenant-branded portal, see only their own tickets, and rate a resolved ticket.

Module 8 — Notifications & Communication

- Email notifications (transactional, templated per tenant), in-app notifications, optional SMS/push.
- Digest options (immediate vs. batched) per user preference.
- **Done when:** all major ticket events have a working notification template, and a user can mute/adjust channels.

Module 9 — Reporting & Analytics

- Tenant-level dashboards: ticket volume, SLA compliance, agent performance, CSAT trends.
- Scheduled report exports (PDF/CSV) via email.
- Custom report builder for enterprise-tier tenants.
- **Done when:** a tenant admin can build and schedule a custom report without engineering help.

Module 10 — Asset Management / CMDB (Enterprise add-on)

- Track hardware/software assets, link assets to tickets (incident against an asset).
- Optional — gated behind an “Enterprise” plan feature flag.
- **Done when:** an asset can be linked to an incident ticket and its history is visible from the asset record.

Module 11 — Integrations & Public API

- REST API (versioned, per-tenant API keys, scoped OAuth), outbound webhooks for ticket events.
- Native integrations: Slack/Teams (notifications + ticket creation from chat), Jira, Microsoft Teams.
- **Done when:** a tenant can generate an API key, create a ticket via API, and receive a webhook on status change.

Module 12 — Roles & Permissions (Tenant-Level RBAC)

- Built-in roles (Admin, Agent, Manager, Read-only) plus custom roles with granular permission sets.
- Team/group structures for ticket routing (e.g., “Billing Team”, “Tier 2 Support”).
- **Done when:** a tenant admin can create a custom role restricted to specific modules and assign it to a user.

5. Super Admin Module (Platform Owner Console)

This is the module **you** (WMX Solutions, the SaaS operator) use — separate app/subdomain from tenant-facing UI, its own auth, and never accessible to tenant users.

5.1 Tenant (Client) Lifecycle Management

- **Create tenant:** manually provision a new client (name, subdomain, plan, initial admin) — used for sales-assisted onboarding.
- **View/search all tenants:** list with status (Trial, Active, Past Due, Suspended, Churned), plan, MRR, seat usage, last login.
- **Suspend / reactivate / offboard tenant:** suspend blocks tenant login instantly (billing failure, ToS violation); offboarding triggers a data export + scheduled deletion per retention policy.
- **Impersonate tenant** (with audit trail): support staff can log in “as” a tenant admin to debug an issue, every impersonation session is logged and visible to the tenant’s audit log too.
- **Tenant-level feature flags:** turn on/off specific modules (e.g., CMDB, custom report builder) per tenant regardless of plan, for pilots or negotiated deals.

5.2 Plans & Billing Administration

- Define/edit plans (pricing, limits: max agents, max tickets/month, storage caps, included modules).
- View subscription status per tenant, manually apply credits/discounts, handle plan overrides.
- Stripe (or chosen billing provider) integration: invoices, payment failures, dunning status, usage-based overage billing.
- Revenue dashboards: MRR/ARR, churn rate, expansion revenue, cohort retention.

5.3 Platform Operations & Health

- Global system health dashboard: API latency, error rates, queue backlogs, DB connection pool usage — per-region if multi-region.
- Per-tenant resource usage: ticket volume, storage used, API call volume — to catch abusive/outlier tenants before they affect others.
- Feature rollout controls: percentage-based rollout of new features across tenants (canary release).

5.4 Security & Compliance

- Global audit log across all tenants (platform-level actions: tenant created/suspended, plan changed, impersonation).
- Data residency controls (if some clients require EU-only data — maps to Neon project region).
- Manage global security policies: password complexity defaults, session timeout defaults, IP allowlisting for platform admin access.
- SOC 2 / GDPR support tooling: data export requests, right-to-be-forgotten execution across tenant data.

5.5 Support & Success Tooling

- Internal notes per tenant account (CS/success team context, renewal risk flags).
- Ability to send platform-wide or targeted announcements/banners (e.g., maintenance windows) to specific tenants.
- Ticket volume alerts to WMX’s own team if a client is approaching plan limits (upsell trigger).

5.6 Platform Admin Roles

Super Admin itself has its own RBAC: Owner, Platform Admin, Support Engineer (impersonation + read), Billing Admin (billing only, no impersonation), Read-only Analyst.

6. How a Client Gets Onboarded and Managed (End-to-End Flow)

1. **Acquisition** — client signs up self-serve (trial) or is created by WMX sales via Super Admin “Create Tenant”.
2. **Provisioning** — a new `Tenants` row is created; a `TenantId` is assigned; default categories, SLA policy, and roles are seeded for that tenant (no new database/schema needed — same Neon TMS database, filtered by `TenantId`).
3. **Setup wizard** — tenant admin sets subdomain, branding, invites agents, configures SLA/business hours.
4. **Go-live** — tenant starts receiving tickets via portal/email/API.
5. **Ongoing management (Super Admin side)** — WMX monitors usage/health, handles billing events (via Stripe webhooks updating `Subscriptions`), responds to support escalations (with impersonation if needed).
6. **Plan changes** — tenant self-upgrades/downgrades from within their own admin settings (Module 2), or WMX applies manual overrides from Super Admin.
7. **Offboarding** — tenant cancels or is suspended for non-payment → grace period → data export offered → scheduled hard delete per data retention policy, all logged in the platform audit log.

7. Non-Functional Requirements

Category	Requirement
Availability	99.9% uptime target for API; graceful degradation (read-only mode) during backend deploys
Scalability	Support 100+ tenants, tens of thousands of tickets/tenant, without cross-tenant performance impact
Security	Encryption in transit (TLS 1.2+) and at rest (Neon + Azure Blob default encryption); tenant data isolation via query filters + RLS
Compliance readiness	Structured audit logging, data export/delete tooling to support SOC 2 and GDPR requests
Performance	P95 API response < 300ms for core ticket operations under normal load
Backups	Neon point-in-time recovery; automated daily backup verification
Observability	Centralized logging/tracing (App Insights) with per-tenant correlation IDs

8. Suggested Implementation Roadmap

Phase 1 — MVP (foundation + core loop): Module 1 (Auth, local only), Module 2 (basic onboarding, no SSO yet), Module 3 (ticketing core, portal + email intake), Module 4 (basic SLA), Module 12 (basic RBAC), Super Admin 5.1 (tenant CRUD only). Goal: one internal test tenant can run a full ticket lifecycle.

Phase 2 — Sellable SaaS: Module 9 (reporting), Module 8 (notifications), Module 6 (KB), billing (Super Admin 5.2, Stripe), Module 5 (automation rules), audit logging (5.4). Goal: can onboard and bill real paying clients.

Phase 3 — Enterprise hardening: SSO (Module 1 completion), custom roles (Module 12 completion), Module 10 (CMDB), Module 11 (public API/integrations), advanced Super Admin (feature flags, impersonation, canary rollout, multi-region). Goal: ready for enterprise clients with compliance requirements.

Phase 4 — Scale-out: read replicas, per-tenant rate limiting refinement, background job isolation, promoted-tenant (dedicated DB) support for the largest clients.

This document is intended to be fed module-by-module into Claude for implementation — reference the module name (e.g. “Module 3 — Ticketing Core”) when starting a build/implementation conversation so scope stays contained.